

AI Control Layer

Build a flexible AI control layer to secure and govern interactions with Agentic AI systems (AI agents, MCP services, LLMs, APIs). This layer will help organizations protect data, apply required guardrails, manage API budgets, and block emerging exploits while maintaining developer speed. Create the ultimate hybrid defense system for the generative AI era!

1. Introduction - organization, context and current state

Modern organizations continuously advance security of their infrastructures while fostering safe and secure technological innovation. Work is being done to ensure that cutting-edge technologies, such as AI, can be adopted safely by developers and employees without compromising data privacy, financial budgets, or more broadly the organizational security. The goal is to bridge the gap between the need of rapid software development and cybersecurity controls and practices.

Nowadays, developers leverage AI and Agentic AI pervasively in their work. These technologies are used to write code, draft documents, and analyze data. Similarly, AI and Agentic AI systems used pervasively to support or fully automate numerous business processes. While this dramatically increases productivity, it also introduces risks that traditional security tools are not necessarily equipped to handle.

Organizations face new problems when adopting AI and Agentic AI:

Among others, agentic systems require more modern and dynamic authentication and access control. Without local enforcement, agentic systems can easily access resources they shouldn't access, impersonate other actors and perform harmful and irreversible actions.

Input validation and output filtering represent another critical risk vector. Unlike traditional applications with structured inputs, AI systems interpret natural language as execution logic, making them highly vulnerable to such attacks as prompt injection. On the output side, a lack of rigorous filtering means AI systems can return sensitive data.

Furthermore, organizations struggle with managing access to memory and resource consumption, especially within autonomous agentic workflows. When agents rely on persistent context or shared memory stores, they can trigger unauthorized data retrievals or get trapped in runaway execution loops. As their behaviour is non-deterministic, they can cause unexpectedly high resource consumption.

To address these risks, organizations need strong and comprehensive controls (often called guardrails) for AI which can be implemented and enforced with a flexible and adaptive control layer (e.g. something that acts as a smart intermediary). Such layer must instantly inspect, redact, or block unsafe interactions in real-time, using a hybrid approach of traditional policy enforcement combined with AI-supported enforcement.

Note: The potential threats listed above represent only examples of risks in agentic systems.

When designing your control layer, you should analyze the AI ecosystem deeply and review available sources (e.g. OWASP) and consider what other controls may need to be implemented to ensure a comprehensive, production-ready defense system which addresses emerging risks and threats.

2. Challenge

Your task is to build a lightweight, flexible AI Control Layer. This layer can be implemented as a gateway, proxy, middleware, or SDK wrapper that intercepts and governs interactions with AI systems. This control layer must enforce security, privacy, and resource controls defined in/provided by a centralized configuration source (e.g. control catalog). The layer should have the ability to generate reporting suitable for security teams as well as management (e.g. via UI or otherwise).

To achieve the optimal balance between speed and deep semantic understanding, your control layer must implement a hybrid defense architecture utilizing both non-AI (deterministic) and AI-based (semantic) controls.

Furthermore, when designing the control layer, think about how to make it resilient enough to manage budgets for both external commercial APIs and locally hosted models, and how it can detect or mitigate known historical attacks on AI infrastructure (i.e. where signatures of such attacks can be fed from some externally managed system).

To prove the reliability and robustness of your solution, you must also deliver a complete, automated testing suite showcasing both positive (allowed) and negative (blocked/redacted) test cases for the controls which you will implement.

3. Expected Outcome

1. AI Control Layer: A functional gateway, proxy, middleware, or SDK wrapper (or other component) that developers can easily integrate into the AI systems (e.g. agent to agent, app to agent, agent to MCP, agent to model, etc communication).
 - a) You can build your own agent OR use an already existing agent to showcase the solution.
 - b) You should provide a simple diagram presenting the architecture of your solution.
2. Sample Configuration: Documented policy file configuring the controls/guardrails and demonstrating different configurable strictness/adherence levels and budget rules for those controls.

3. Simple Interactive Dashboard: UI displaying controls, overall security posture, blocked threats, and other metrics (e.g. resource consumption/cost).
 4. Executable Test Suite: A ready to run test-suite which can verify implementation of your controls, including budget limits and exploit mitigation.
-

4. Formal Requirements

1. Centralized Policy Engine: A single config source (e.g. file/system) managing controls, sensitivity thresholds (Block vs Redact or adherence %), allowed LLM models, and resource/financial budgets.
 2. Controls/Guardrails:
 1. Deterministic (Non-AI): e.g. pattern matching (detecting PII or secrets), checking of authentication or access requirements, etc.
 2. Semantic (AI-Based): where possible, consider using AI-based solutions/model to secure interaction with the AI systems.
 3. Budget and Resource Governance: When designing the control layer, think about how to enforce budget limits (e.g. resource access, compute time or token spend for access to LLMs).
 4. Historical Attack Mitigation: think about how to detect and block patterns associated with successful historical exploits on AI systems, such as malicious code execution, unsafe deserialization, or supply-chain exploits targeting model repositories.
 5. Security Reporting & Auditing: real-time metrics (blocked interactions, budget usage) for management and exportable audit logs designed for security teams to analyze threats, policy violations, and system usage; can be implemented as a dedicated dashboard or otherwise.
 6. Self-Testing Suite: Automated test suite verifying both positive (allowed) and negative (blocked) cases for the controls.
-

5. Technical Requirements

Teams have complete flexibility in choosing their technology stack, whether they build the solution from scratch using languages like Go, Rust, or Python, or build upon existing open-source tools (make sure you check the license of these tools). For the actual agents, LLMs and applications, which will use your control layer, you can use pre-existing tooling (agents, applications and other unrelated components will not be subject to assessment).

6. Testing and/or Validation Approach

The evaluation relies primarily on the teams own deliverables and spontaneous, zero-preparation actions. Judges will execute the automated test suite provided by the team (so make sure it allows to test the implemented controls). The test suite must contain both positive and negative test cases to prove the system works as intended. Judges may interactively test the running control layer in real time using spontaneous, ad-hoc prompts and observing how the system reacts. The judges may modify the configuration files/feeds to your AI Control Layer (e.g. changing rules, removing controls, adjusting thresholds) to understand how the control layer behaves with new configuration (how changes are reflected, can they adjust in real-time, etc). You should be able to produce performance telemetry as it may be used for evaluation. The judges will also review the overall architecture, dashboards and logging information which can be provided for management and security teams.

7. Available Resources

No pre-packaged datasets, proprietary APIs, or specific hardware resources are provided for this challenge. This is intentional to give teams complete architectural freedom and to avoid the constraints of static synthetic data. Teams are expected to use publicly available open-source libraries where necessary and local models (such as those run via Ollama), and their own self-created test prompts to demonstrate and validate their control layer. No subscriptions on paid services (e.g. OpenAI, Anthropic, Copilot, etc) will be provided for this challenge so ensure that you can design, build and run the entire system on your own setup.

8. Evaluation Criteria

To ensure a balanced assessment of both technical execution and practical utility, the projects will be evaluated based on the following criteria:

- Robustness of the Solution and Quality of Guardrails (30%)
 - Architecture and Performance Efficiency (20%)
 - Security Reporting (20%)
 - Completeness of the Self-Testing Suite (15%)
 - Practical Implementability and Scalability (15%)
-